

CONTRATO MÍNIMO

Desenvolvimento de uma API REST para Sistema de Estacionamento

Documento de requisitos e contrato de API para fins de estudo

MVP sem autenticação, pagamentos ou integrações externas.
Foco em REST, DTOs, JPA, validações, regras de negócio, transações e tratamento de erros.

Versão 1.0 - Junho de 2026

Visão geral do documento

Finalidade. Este documento define um contrato mínimo para orientar a implementação de uma API de estacionamento. Ele funciona como uma especificação anterior ao código e deve ser usado como referência para modelagem, desenvolvimento e testes.

Abordagem. O escopo foi reduzido para permitir que o projeto seja concluído por etapas, sem perder elementos importantes de uma aplicação corporativa: recursos REST, DTOs, persistência, validações, regras de negócio, transações, padronização de erros e critérios de aceite.

Conteúdo

Seção	Assunto
1	Objetivo
2	Escopo do MVP
3	Recursos principais
4	Endpoints
5	Regra de cobrança
6	Estrutura padrão de erros
7	DTOs sugeridos
8	Modelo de dados mínimo
9	Critérios de aceite
10	Ordem recomendada de implementação

1. Objetivo

Desenvolver uma API REST para controlar as operações essenciais de um estacionamento:

- cadastro de vagas;
- entrada de veículos;
- saída de veículos;
- consulta de veículos estacionados;
- cálculo do valor da permanência.

O sistema não terá autenticação neste primeiro momento.

2. Escopo do MVP

A API deverá permitir:

1. cadastrar uma vaga;

2. listar as vagas;
3. registrar a entrada de um veículo;
4. registrar a saída de um veículo;
5. consultar uma estadia pelo identificador;
6. listar estadias ativas.

Fora do escopo

Não fazem parte desta primeira versão:

- cadastro de clientes;
- mensalistas;
- reservas;
- pagamentos;
- emissão de nota fiscal;
- autenticação;
- diferentes tabelas de preço;
- integração com cancelas ou câmeras.

3. Recursos principais

3.1 Vaga

Representa um espaço físico do estacionamento.

```
{
  "id": "f4e8ea86-0c84-47ce-9a98-7df137af071c",
  "numero": "A-01",
  "tipo": "CARRO",
  "status": "LIVRE"
}
```

Campo	Descrição
id	Identificador único da vaga.
numero	Identificação visível da vaga, por exemplo A-01.
tipo	Tipo de veículo aceito: CARRO ou MOTO.
status	Situação atual: LIVRE ou OCUPADA.

Valores permitidos

Propriedade	Valores
tipo	CARRO, MOTO
status	LIVRE, OCUPADA

3.2 Estadia

Representa o período em que um veículo permaneceu no estacionamento.

```
{
  "id": "905ac572-62f1-4835-8ec5-eaa25fd954c0",
  "placa": "ABC1D23",
  "tipoVeiculo": "CARRO",
  "vaga": {
    "id": "f4e8ea86-0c84-47ce-9a98-7df137af071c",
    "numero": "A-01"
  },
  "dataEntrada": "2026-06-28T10:00:00",
  "dataSaida": null,
  "valor": null,
  "status": "ATIVA"
}
```

Campo	Descrição
id	Identificador único da estadia.
placa	Placa do veículo.
tipoVeiculo	Tipo do veículo: CARRO ou MOTO.
vaga	Resumo da vaga utilizada.
dataEntrada	Data e hora registradas pelo servidor.
dataSaida	Data e hora da saída; nula enquanto ativa.
valor	Valor final; nulo enquanto ativa.
status	ATIVA ou FINALIZADA.

4. Endpoints

Base URL: /api/v1

4.1 Cadastrar vaga

POST /api/v1/vagas

Requisição

```
{
  "numero": "A-01",
  "tipo": "CARRO"
}
```

Resposta de sucesso

```

HTTP/1.1 201 Created
Location: /api/v1/vagas/f4e8ea86-0c84-47ce-9a98-7df137af071c

{
  "id": "f4e8ea86-0c84-47ce-9a98-7df137af071c",
  "numero": "A-01",
  "tipo": "CARRO",
  "status": "LIVRE"
}

```

Validações e regras

- O número da vaga é obrigatório.
- O tipo da vaga é obrigatório.
- Não pode existir outra vaga com o mesmo número.
- Toda nova vaga deve ser criada com status LIVRE.

Status	Situação
201 Created	Vaga criada.
400 Bad Request	Dados inválidos.
409 Conflict	Número de vaga já cadastrado.

4.2 Listar vagas

GET /api/v1/vagas

Filtros opcionais:

```

GET /api/v1/vagas?status=LIVRE
GET /api/v1/vagas?tipo=CARRO
GET /api/v1/vagas?tipo=CARRO&status=LIVRE

```

Resposta

```

HTTP/1.1 200 OK

[
  {
    "id": "f4e8ea86-0c84-47ce-9a98-7df137af071c",
    "numero": "A-01",
    "tipo": "CARRO",
    "status": "LIVRE"
  },
  {
    "id": "c958934d-95e8-423b-883f-46b23adb874b",
    "numero": "A-02",
    "tipo": "CARRO",
    "status": "OCUPADA"
  }
]

```

```
]
```

Caso nenhuma vaga seja encontrada, a API deve retornar 200 OK com uma lista vazia:

```
[]
```

4.3 Registrar entrada

POST /api/v1/estadias

Requisição

```
{
  "placa": "ABC1D23",
  "tipoVeiculo": "CARRO",
  "vagaId": "f4e8ea86-0c84-47ce-9a98-7df137af071c"
}
```

Resposta de sucesso

```
HTTP/1.1 201 Created
Location: /api/v1/estadias/905ac572-62f1-4835-8ec5-eaa25fd954c0

{
  "id": "905ac572-62f1-4835-8ec5-eaa25fd954c0",
  "placa": "ABC1D23",
  "tipoVeiculo": "CARRO",
  "vaga": {
    "id": "f4e8ea86-0c84-47ce-9a98-7df137af071c",
    "numero": "A-01"
  },
  "dataEntrada": "2026-06-28T10:00:00",
  "dataSaida": null,
  "valor": null,
  "status": "ATIVA"
}
```

A data de entrada deve ser definida pelo servidor.

Validações e regras

- A placa é obrigatória.
- A placa deve possuir sete caracteres alfanuméricos.
- O tipo do veículo é obrigatório.
- A vaga deve existir.
- A vaga deve estar livre.
- O tipo da vaga deve corresponder ao tipo do veículo.
- Um veículo não pode possuir mais de uma estadia ativa.
- Depois da entrada, a vaga deve passar para OCUPADA.

Status	Situação
201 Created	Entrada registrada.

Status	Situação
400 Bad Request	Dados inválidos.
404 Not Found	Vaga não encontrada.
409 Conflict	Vaga ocupada ou veículo já estacionado.

4.4 Consultar estadia

GET /api/v1/estadias/{id}

Exemplo:

```
GET /api/v1/estadias/905ac572-62f1-4835-8ec5-eea25fd954c0
```

Resposta

```
HTTP/1.1 200 OK

{
  "id": "905ac572-62f1-4835-8ec5-eea25fd954c0",
  "placa": "ABC1D23",
  "tipoVeiculo": "CARRO",
  "vaga": {
    "id": "f4e8ea86-0c84-47ce-9a98-7df137af071c",
    "numero": "A-01"
  },
  "dataEntrada": "2026-06-28T10:00:00",
  "dataSaida": null,
  "valor": null,
  "status": "ATIVA"
}
```

Status	Situação
200 OK	Estadia encontrada.
404 Not Found	Estadia não encontrada.

4.5 Listar estadias

GET /api/v1/estadias

Filtros opcionais:

```
GET /api/v1/estadias?status=ATIVA
GET /api/v1/estadias?status=FINALIZADA
GET /api/v1/estadias?placa=ABC1D23
```

Resposta

```
HTTP/1.1 200 OK

[
  {
    "id": "905ac572-62f1-4835-8ec5-eea25fd954c0",
    "placa": "ABC1D23",
    "tipoVeiculo": "CARRO",
    "vaga": {
      "id": "f4e8ea86-0c84-47ce-9a98-7df137af071c",
      "numero": "A-01"
    },
    "dataEntrada": "2026-06-28T10:00:00",
    "dataSaida": null,
    "valor": null,
    "status": "ATIVA"
  }
]
```

4.6 Registrar saída

POST /api/v1/estadias/{id}/saida

Não é necessário enviar corpo na requisição.

```
POST /api/v1/estadias/905ac572-62f1-4835-8ec5-eea25fd954c0/saida
```

Resposta

```
HTTP/1.1 200 OK

{
  "id": "905ac572-62f1-4835-8ec5-eea25fd954c0",
  "placa": "ABC1D23",
  "tipoVeiculo": "CARRO",
  "vaga": {
    "id": "f4e8ea86-0c84-47ce-9a98-7df137af071c",
    "numero": "A-01"
  },
  "dataEntrada": "2026-06-28T10:00:00",
  "dataSaida": "2026-06-28T12:20:00",
  "valor": 20.00,
  "status": "FINALIZADA"
}
```

Regras

- A estadia deve existir.
- A estadia deve estar com status ATIVA.
- A data de saída deve ser definida pelo servidor.
- Depois da saída, a estadia deve ficar FINALIZADA.
- Depois da saída, a vaga deve voltar para LIVRE.
- Uma estadia finalizada não pode ser finalizada novamente.

Status	Situação
200 OK	Saída registrada.
404 Not Found	Estadia não encontrada.
409 Conflict	Estadia já finalizada.

5. Regra de cobrança

Para simplificar o estudo, será utilizada uma tabela única:

- Primeira hora ou fração: R\$ 10,00.
- Cada hora adicional ou fração: R\$ 5,00.

Permanência	Valor
30 minutos	R\$ 10,00
1 hora	R\$ 10,00
1 hora e 20 minutos	R\$ 15,00
2 horas	R\$ 15,00
2 horas e 20 minutos	R\$ 20,00

Exemplo de cálculo

```
Entrada: 10:00
Saída: 12:20

Tempo cobrado: 3 horas
Primeira hora: R$ 10,00
Duas horas adicionais: 2 x R$ 5,00

Total: R$ 20,00
```

Observação técnica: para valores monetários em Java, utilize BigDecimal, e não double.

6. Estrutura padrão de erros

Todos os erros devem seguir um formato consistente, facilitando o consumo da API pelo front-end.

Erro de regra de negócio

```
{
  "status": 409,
  "erro": "CONFLITO",
  "mensagem": "A vaga A-01 já está ocupada.",
}
```

```
{
  "timestamp": "2026-06-28T10:05:00",
  "path": "/api/v1/estadias",
  "errosCampos": []
}
```

Erro de validação

```
{
  "status": 400,
  "erro": "REQUISICAO_INVALIDA",
  "mensagem": "Existem campos inválidos na requisição.",
  "timestamp": "2026-06-28T10:05:00",
  "path": "/api/v1/estadias",
  "errosCampos": [
    {
      "campo": "placa",
      "mensagem": "A placa é obrigatória."
    }
  ]
}
```

7. DTOs sugeridos

Os exemplos abaixo utilizam records, adequados para projetos com versões modernas do Java.

7.1 CriarVagaRequest

```
public record CriarVagaRequest(
    String numero,
    TipoVaga tipo
) {
}
```

7.2 VagaResponse

```
public record VagaResponse(
    UUID id,
    String numero,
    TipoVaga tipo,
    StatusVaga status
) {
}
```

7.3 CriarEstadiaRequest

```
public record CriarEstadiaRequest(
    String placa,
    TipoVeiculo tipoVeiculo,
    UUID vagaId
) {
}
```

7.4 VagaResumoResponse

```
public record VagaResumoResponse(
    UUID id,
    String numero
) {
}
```

7.5 EstadiaResponse

```
public record EstadiaResponse(
    UUID id,
    String placa,
    TipoVeiculo tipoVeiculo,
    VagaResumoResponse vaga,
    LocalDateTime dataEntrada,
    LocalDateTime dataSaida,
    BigDecimal valor,
    StatusEstadia status
) {
}
```

8. Modelo de dados mínimo

8.1 Tabela vaga

Coluna	Regra
id	Chave primária.
numero	Único e obrigatório.
tipo	Obrigatório.
status	Obrigatório.

8.2 Tabela estadia

Coluna	Regra
id	Chave primária.
placa	Obrigatória.
tipo_veiculo	Obrigatório.
vaga_id	Chave estrangeira para vaga.
data_entrada	Obrigatória.

Coluna	Regra
data_saida	Opcional enquanto a estadia estiver ativa.
valor	Opcional enquanto a estadia estiver ativa.
status	Obrigatório.

8.3 Relacionamento

Vaga 1 ----- N Estadia

Uma vaga pode possuir várias estadias ao longo do tempo, mas apenas uma estadia ativa por vez.

9. Critérios de aceite

O projeto estará funcional quando todos os comportamentos abaixo forem atendidos:

1. For possível cadastrar vagas.
2. Não for possível cadastrar duas vagas com o mesmo número.
3. For possível registrar a entrada em uma vaga livre.
4. Não for possível registrar entrada em uma vaga ocupada.
5. Não for possível registrar duas entradas ativas para a mesma placa.
6. Ao registrar entrada, a vaga ficar ocupada.
7. Ao registrar saída, o valor ser calculado.
8. Ao registrar saída, a vaga voltar a ficar livre.
9. Não for possível finalizar uma estadia duas vezes.
10. Erros de validação e de negócio retornarem respostas padronizadas.

10. Ordem recomendada de implementação

1. Criar os enums.
2. Criar a entidade Vaga.
3. Criar o CRUD mínimo de vagas.
4. Criar a entidade Estadia.
5. Implementar o registro de entrada.
6. Implementar a consulta de estadias.
7. Implementar o registro de saída.
8. Implementar o cálculo do valor.
9. Criar o GlobalExceptionHandler.
10. Adicionar testes unitários e testes de integração.

Resultado esperado do estudo. Ao concluir o contrato, o projeto terá exercitado REST, DTOs, JPA, relacionamentos, validações, regras de negócio, exceções, ResponseEntity, consultas, transações e testes.